



TEC

IC-4700 Lenguajes de Programación

Proyecto Programado #4: Galactic Colonies

Profesor: Ing. Allan Rodríguez Dávila

Integrantes:

Tayler Wynta Rodríguez | 2024143103

Maikel Flores Navarro | 2024148346

I Semestre 2026

CONTENIDOS

1. MANUAL DE USUARIO	4
1.1 INSTRUCCIONES DE EJECUCIÓN	4
1.2 PRUEBAS DE FUNCIONALIDAD	4
1.2.1 Prueba de creación de partida	4
1.2.2 Prueba de creación de partida	5
1.2.3 Prueba de unirse a partida	6
1.2.4 Prueba de inicio de partida	8
1.2.5 Prueba de producción de recursos.....	8
1.2.6 Prueba de construcción	9
1.2.7 Prueba de movilización de flotas	13
1.2.8 Prueba de conquista.....	15
1.2.9 Prueba de sincronización en tiempo real.....	17
1.2.10 Prueba de fin de partida	18
1.2.11 Prueba de ranking	19
3. DESCRIPCIÓN DEL PROBLEMA	20
4. DISEÑO DEL PROGRAMA.....	21
4.1 Diagrama de paquetes.....	21
4.2 Diagrama de distribución.....	22
4.3 Diagrama de comunicación	23
Backend	23
Frontend	24
Comunicación	24
6. ANÁLISIS DE RESULTADOS	24
6.1 Objetivos alcanzados	24
6.2 Objetivos no alcanzados	25
6.3 Razones por las cuales no se alcanzaron los objetivos.....	25
7. DESCRIPCIÓN DE LOS PRINCIPALES ALGORITMOS.....	26
7.1 Carga de galaxias	26
7.2 Producción de recursos	26

7.3 Construcción de infraestructura	27
7.4 Movilización de flotas	27
7.5 Conquista de sistemas	28
7.6 Cálculo de ranking	28
8. BITÁCORA.....	29

1. MANUAL DE USUARIO

1.1 INSTRUCCIONES DE EJECUCIÓN

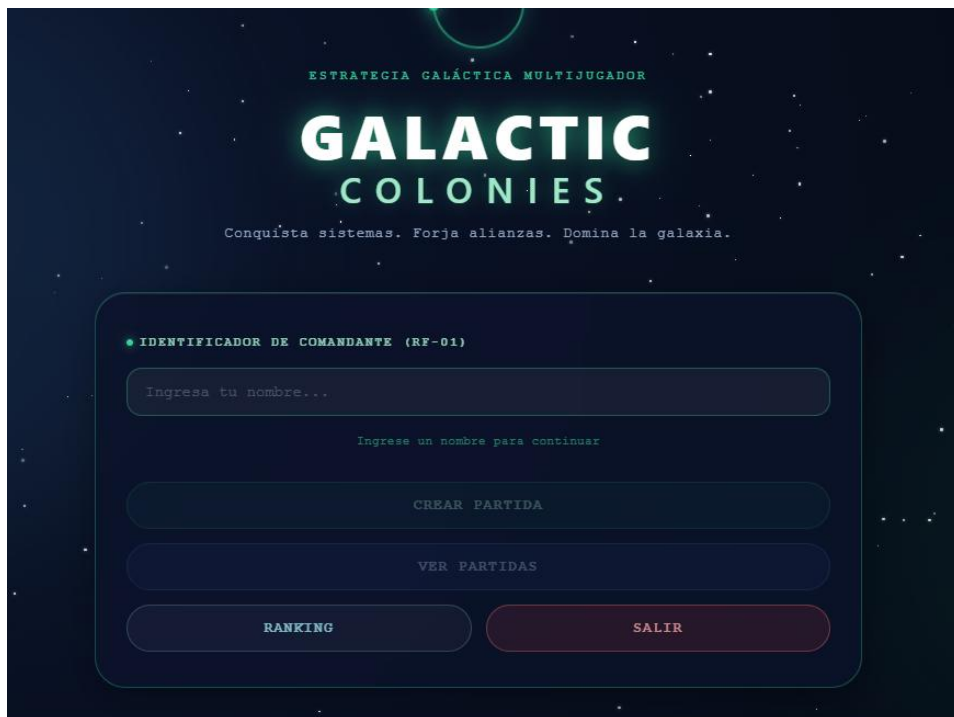
Ingresa al deploy del proyecto con este enlace:

<https://colonias-galacticas-wggq.onrender.com>

1.2 PRUEBAS DE FUNCIONALIDAD

1.2.1 Prueba de creación de partida

Una vez ingresado al sistema se mostrará la siguiente pantalla principal:



El usuario tendrá que ingresar un nombre de usuario válido (no se pueden usar nombres que ya estén registrados dentro de una sala)



● IDENTIFICADOR DE COMANDANTE (RF-01)

Tayler OK

CREAR PARTIDA

VER PARTIDAS

RANKING SALIR

1.2.2 Prueba de creación de partida

Tras ingresar un nombre válido y presionar “crear partida” se mostrará esta ventana



CONFIGURAR NUEVA PARTIDA

Ajusta los parámetros para inicializar el nexo galáctico.

NOMBRE DE LA SALA

Ej: Flota Alfa...

GALAXIA MÁX. JUGADORES

Nebulosa Orion ✓ 4

TIEMPO LÍMITE (MIN) RECURSOS INICIALES

45 Bajo (100M, 50E, 20C) ✓

CANCELAR INICIALIZAR

Rellenar: Nombre de la sala, Galaxia, Max. Jugadores, Tiempo Limite y Recursos iniciales

CONFIGURAR NUEVA PARTIDA

Ajusta los parámetros para inicializar el nexo galáctico.

NOMBRE DE LA SALA

Flota Gamma

GALAXIA

Galaxia Andrómeda

MÁX. JUGADORES

2

TIEMPO LÍMITE (MIN)

20

RECURSOS INICIALES

Normal (300M, 150E, 50C)

CANCELAR

INICIALIZAR

Tras crear la partida y presionar “inicializar” será redirigido al lobby como anfitrión de la sala:

CONFIGURACIÓN DE PARTIDA

Galaxia	Galaxia Andrómeda
Duración	20 min
Recursos	
ID Partida	76625552

SALA DE ESPERA

1 / 2

P1



Maikel (s6)

★ ANFITRIÓN

LISTO

P2

Esperando jugador...

... Esperando 1 jugador(es) más para iniciar...

COMUNICACIONES

Canal abierto. ¡Di algo, comandante!

Mensaje... 

1.2.3 Prueba de unirse a partida

Una vez un usuario haya creado una sala, esta saldrá disponible en la opción “Ver Partidas”

● IDENTIFICADOR DE COMANDANTE (RF-01)

Tayler

OK

CREAR PARTIDA

VER PARTIDAS

RANKING

SALIR

PARTIDAS EN TIEMPO REAL

[RF-05/RF-06] Canal WebSocket activo. Selecciona una partida disponible.

Nombre	Galaxia	Jugadores	Duración	Estado	Acción
Flota Gamma	Galaxia Andrómeda	1 / 2	20 min	Abierta	UNIRSE

VOLVER AL MENÚ

En donde saldrán todas las partidas disponibles que no estén inicializadas ni llenas. Si el usuario presiona “Unirse” se unirá al lobby de la partida.

⊙ CONFIGURACIÓN DE PARTIDA

Galaxia

Galaxia Andrómeda

Duración

20 min

Recursos

ID Partida

76625552

👤 SALA DE ESPERA

2 / 2

P1

Maikel

★ ANFITRIÓN

LISTO

P2

Tayler (SG)

LISTO

▶ ESPERANDO AL ANFITRIÓN

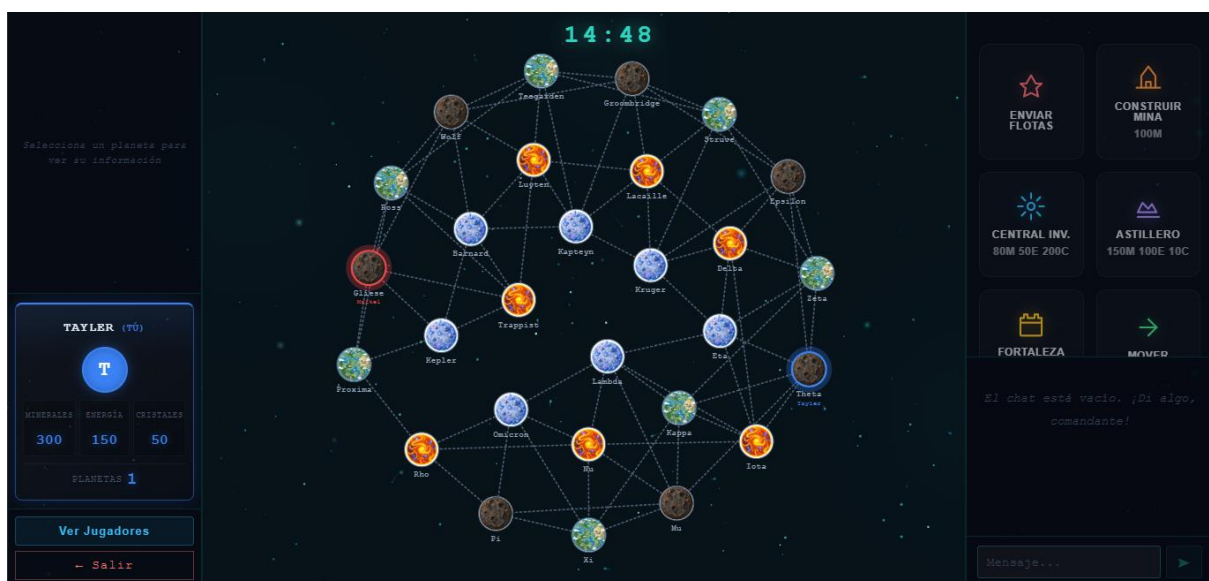
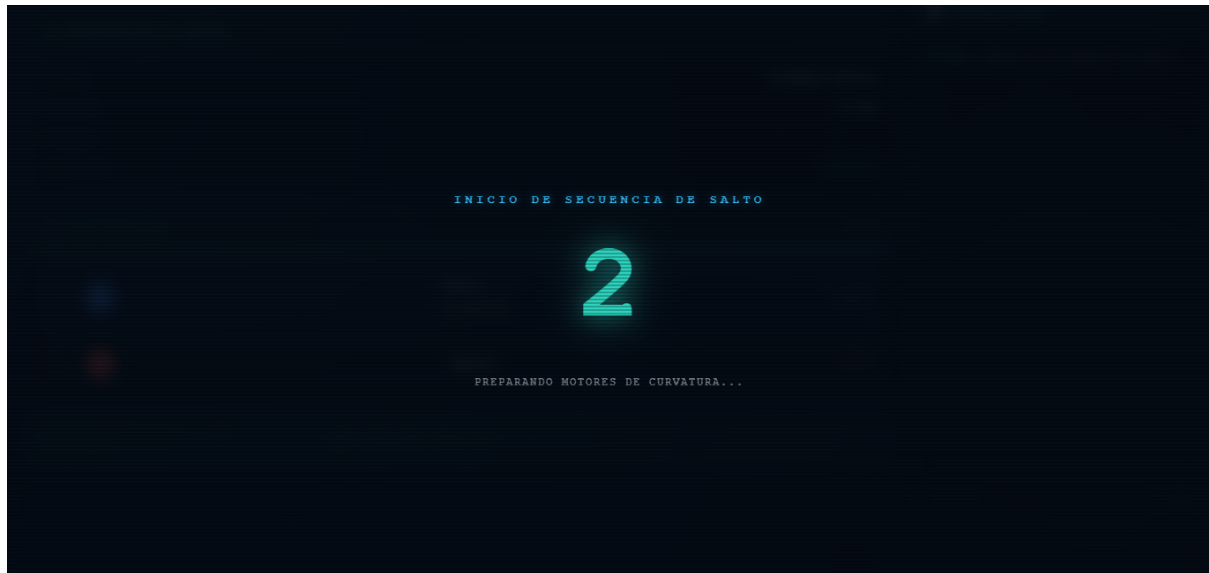
🗨 COMUNICACIONES

Canal abierto. ¡Di algo, comandante!

Mensaje... ▶

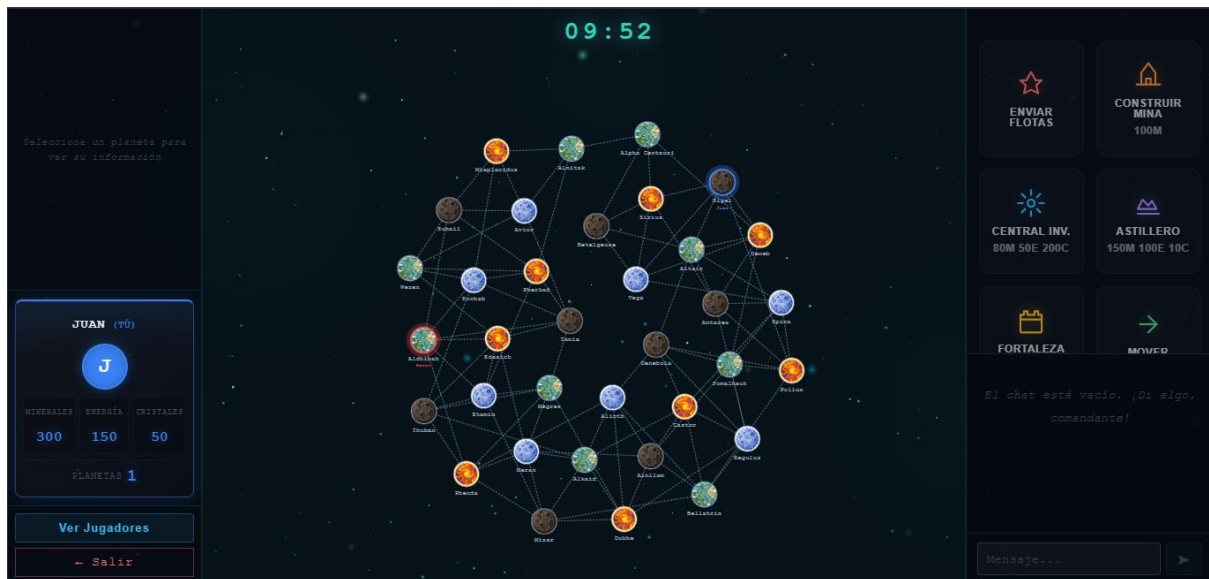
1.2.4 Prueba de inicio de partida

Si el anfitrión presiona U o el botón de iniciar partida, se iniciará una cuenta regresiva de 3 segundos antes de empezar.

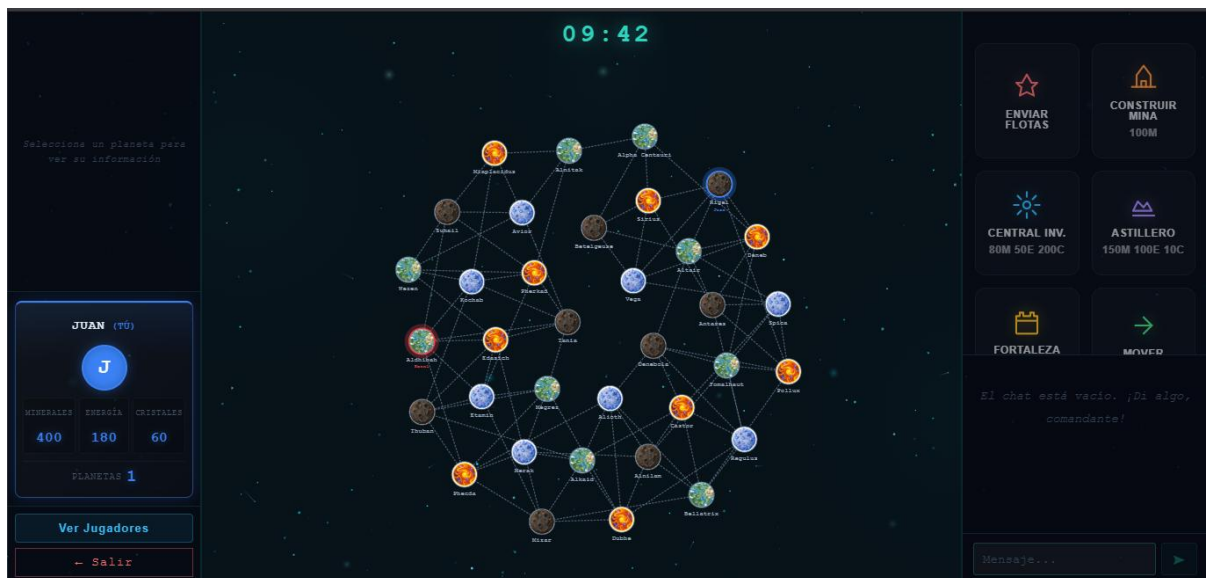


1.2.5 Prueba de producción de recursos

Esta partida fue configurada con 10 minutos de juego y generar recursos cada 17 segundos, se puede apreciar la generación de recursos en las siguientes imágenes:

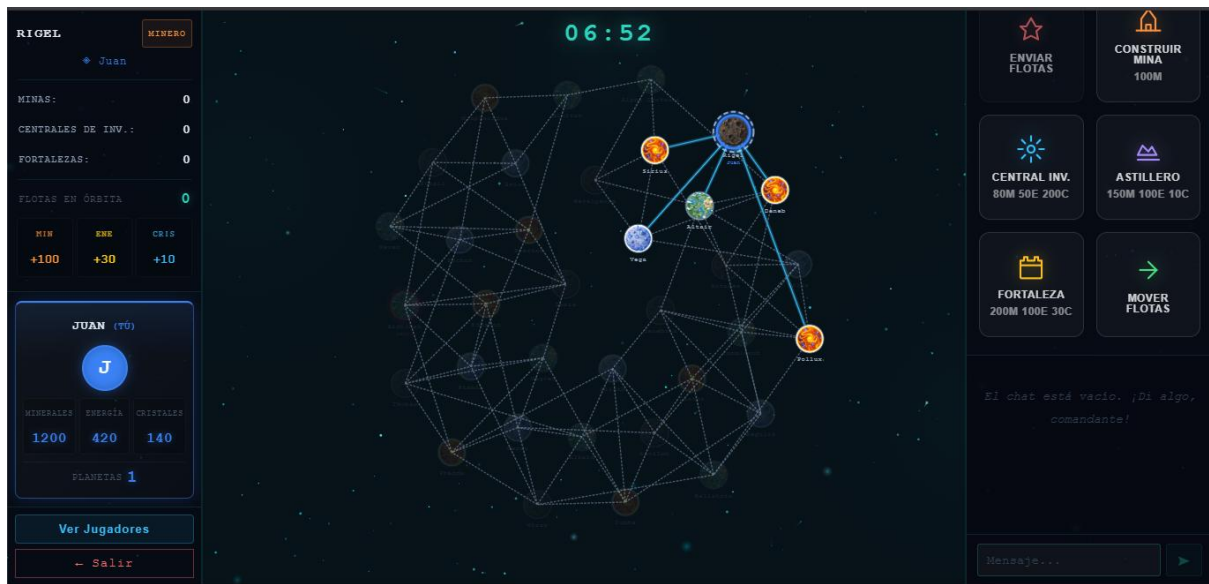


Después de que transcurran los 17 segundos se generaron los recursos, y se actualizaron



1.2.6 Prueba de construcción

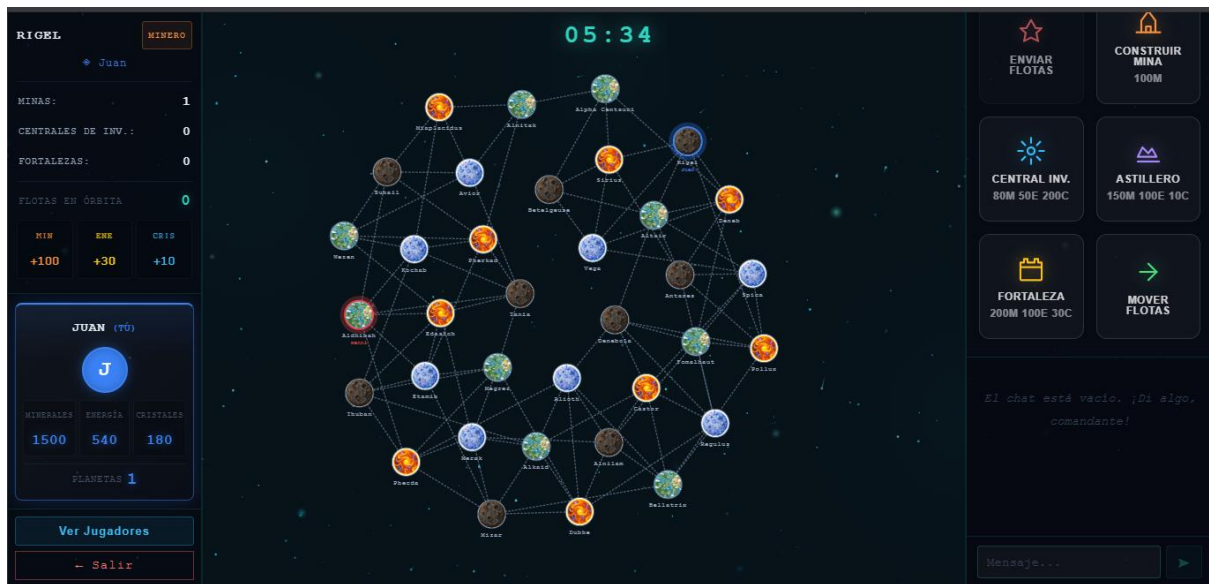
Estos son los recursos que se disponían antes de seleccionar un planeta y realizar la construcción, para realizar la construcción en un planeta, hay que seleccionarlo y luego elegir la construcción deseada en el panel de acciones derecho.



Una vez seleccionado el planeta, se elegirá la acción a realizar en él:

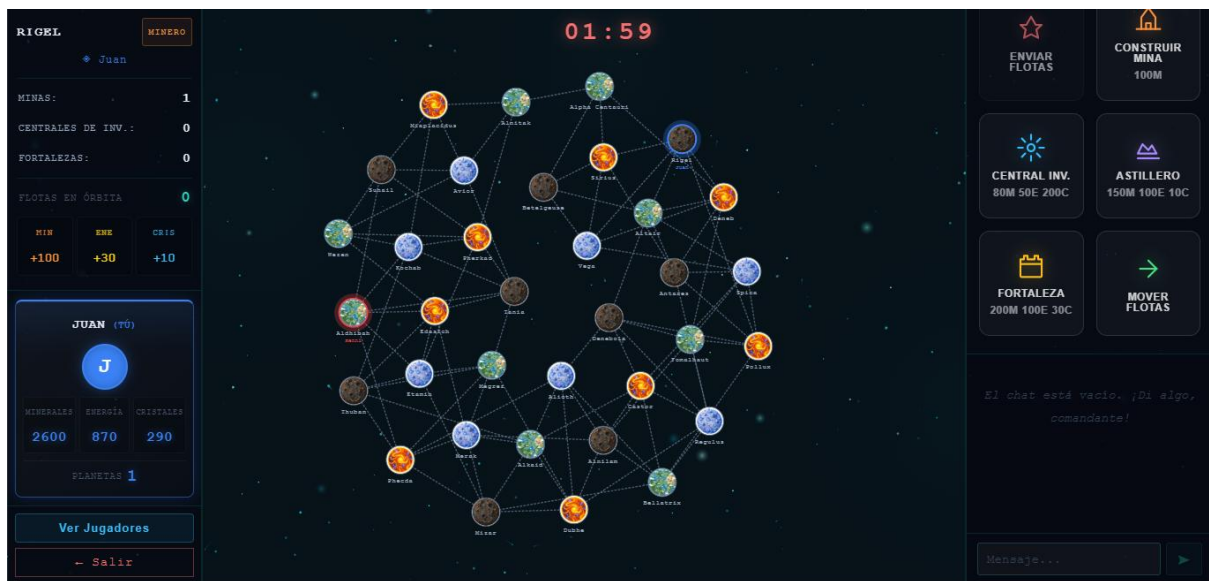
- A) **Construir mina:** esta opción construye una mina en un planeta seleccionado y se descuentan los recursos de forma dinámica.





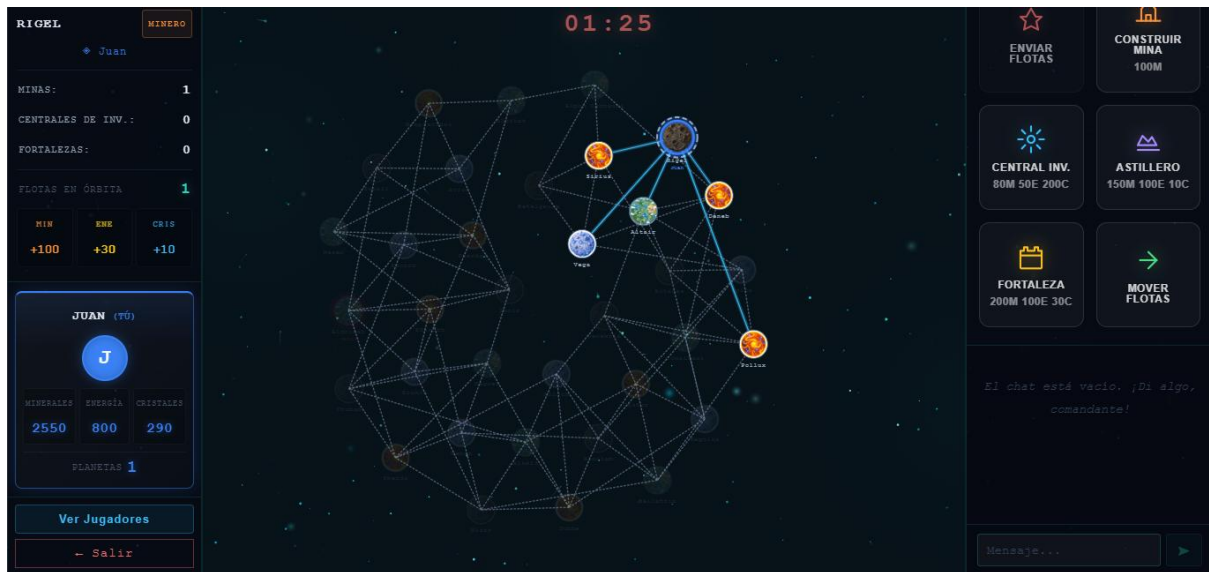
B) **Construir central de investigación:** Esta opción construye una central de investigación en un planeta seleccionado y se descuentan los recursos de forma dinámica. (Dando un mensaje de error si el jugador no posee los recursos necesarios)



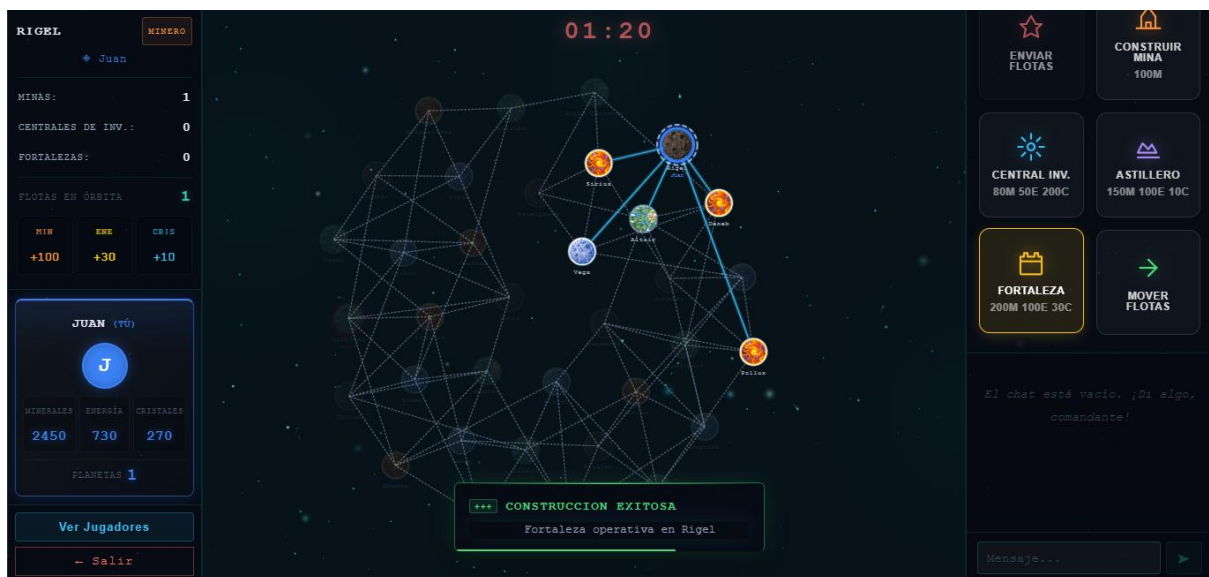


C) **Construir astillero:** Esta opción construye un astillero en un planeta seleccionado y se descuentan los recursos de forma dinámica.



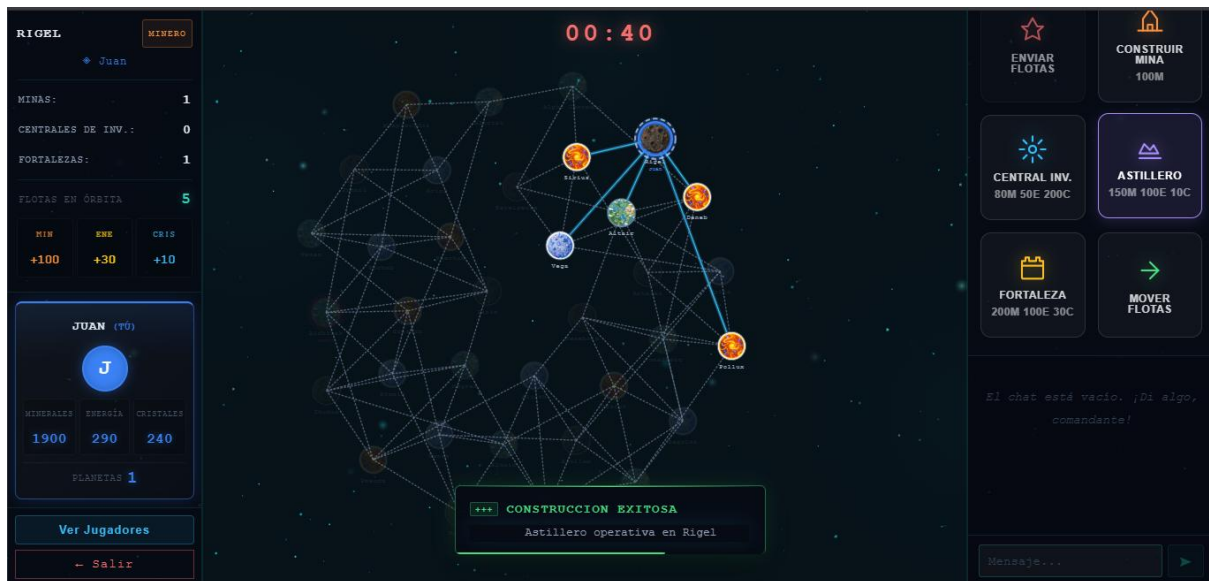


D) **Construir fortaleza:** Esta opción construye una fortaleza en un planeta seleccionado y se descuentan los recursos de forma dinámica.

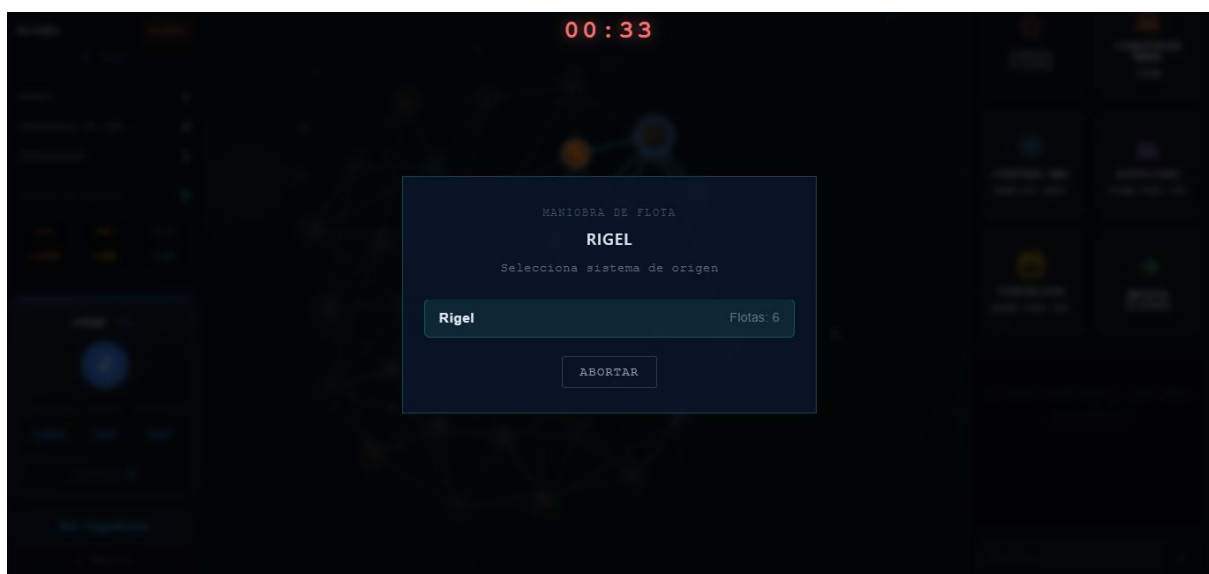


1.2.7 Prueba de movilización de flotas

Para movilizar flotas, se necesita de la creación previa de un astillero, cuando ya se tenga, se selecciona el planeta que lo contenga:



Y se presiona el botón “Mover Flotas” en el panel derecho, mostrando una lista de los planetas conquistados y sus flotas, primero seleccionando el planeta de origen y luego el de destino. (si el planeta seleccionado no tiene flotas, se mostrará un mensaje de error)

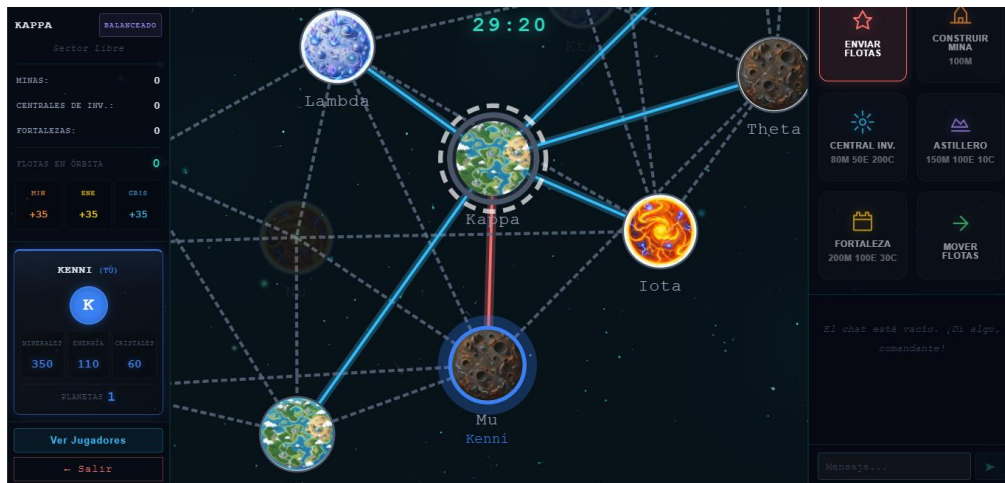


Una vez movidas las flotas deseadas al planeta deseado, se actualizará en tiempo real la cantidad de flotas en órbita de cada planeta, y, si fue posible realizar el movimiento de flotas si se encontró una ruta válida, saldrá un mensaje de éxito.



1.2.8 Prueba de conquista

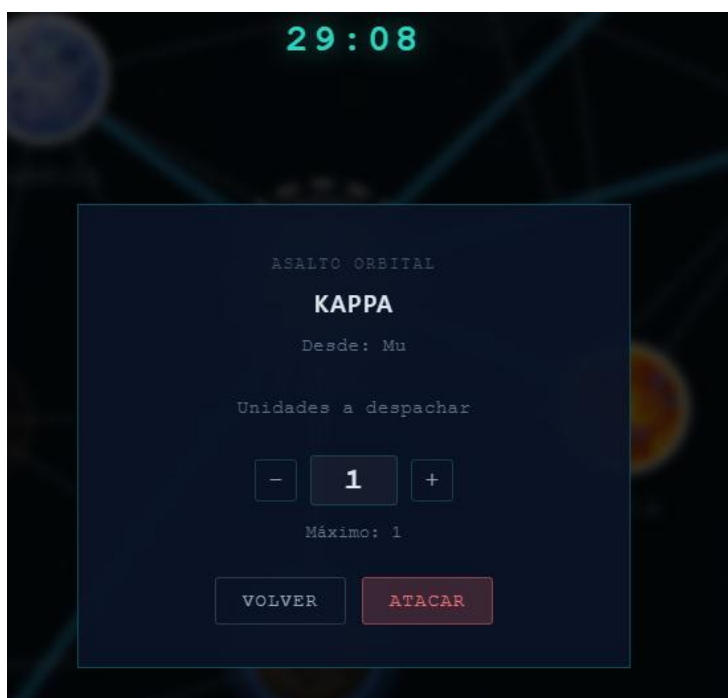
Para conquistar un planeta, hay que seleccionar un planeta sin dueño o uno ajeno, debe de haber una ruta disponible indirecta o directamente con el planeta de origen del cual se está realizando el ataque. (Si no se encuentra una ruta válida, o el ataque fue repelido, se mostrará un mensaje de error respectivamente). Tras seleccionar el planeta de destino (el que se quiere conquistar), presionar en el panel de acciones “Enviar flotas”



Tras abrir el panel de conquista, se mostrará una lista de todos los planetas conquistados y sus flotas, donde se seleccionará el planeta origen del ataque para movilizar sus flotas y realizar el ataque



Tras seleccionar el planeta de origen, se le pedirá al usuario la cantidad de flotas a despachar para la conquista. (Si un sistema planetario no tiene flotas disponibles, se mostrará un mensaje de error).

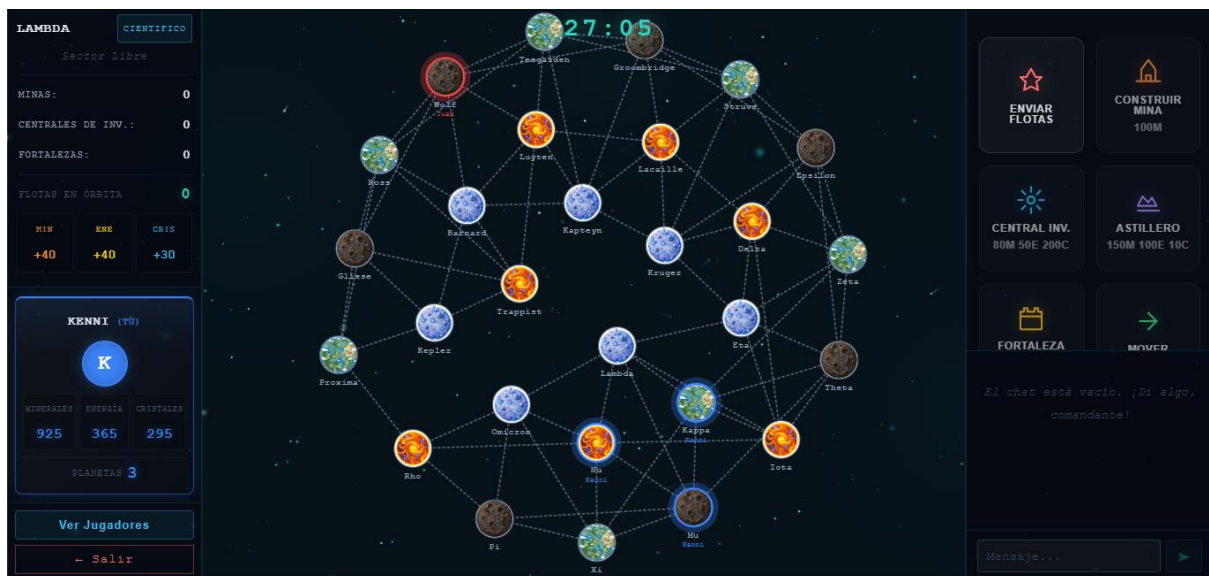


Si el ataque fue realizado con éxito, la conquista se llevará a cabo, haciendo que el planeta ahora sea suyo.

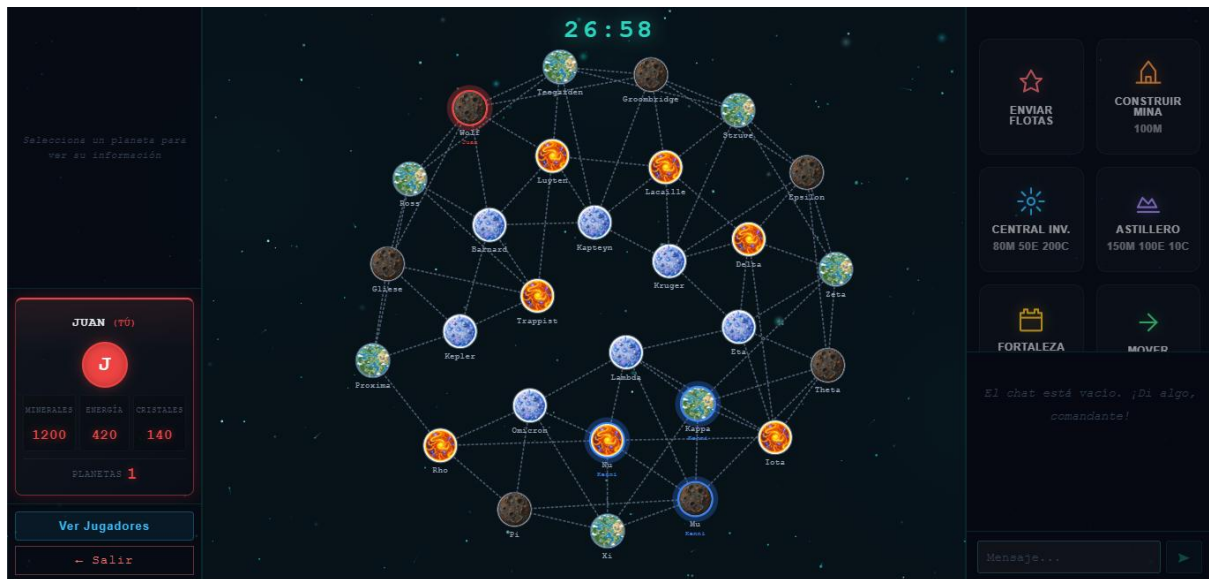


1.2.9 Prueba de sincronización en tiempo real

Con 2 pestañas abiertas del deploy, se mostrará al Jugador 1: Kenni y al Jugador 2: Juan, en la primera imagen se muestra la ventana de Kenni



Se puede ver la ventana del jugador 1, con su mapa galáctico.



Y este es el mapa galáctico del jugador 2, sincronizado correctamente con el estado actual de juego



Si se presiona “Ver jugadores” se podrá ver el estado en tiempo real de los jugadores

1.2.10 Prueba de fin de partida

Fin de partida por derrota



Fin de partida por victoria



1.2.11 Prueba de ranking

Tras finalizar una partida, se guardará el nombre del ganador y se mostrará en el ranking del menú principal

SALÓN DE LA INFAMIA GALÁCTICA

[RF-23] Historial persistido en base de datos central planetaria.

ID Partida	Fecha	Ganador	Sistemas	Recursos	Galaxia	Tiempo
partida_1782177634306	22/6/2026	Kenni	1	M: 1315, E: 1165, C: 1065	Galaxia Andrómeda	600s
partida_1782176885432	22/6/2026	Maikel	21	M: 5235, E: 3575, C: 1915	Triángulo Austral	363s
partida_1782175122261	22/6/2026	Maikel	2	M: 7399, E: 8299, C: 9839	Nebulosa Orion	48s
partida_1781992417897	20/6/2026	Maikel	23	M: 3429, E: 5529, C: 10649	Nebulosa Orion	230s
partida_1781983156553	20/6/2026	afasffeg	1	M: 11159, E: 11159, C: 10869	Nebulosa Orion	600s

CERRAR HISTORIAL

3. DESCRIPCIÓN DEL PROBLEMA

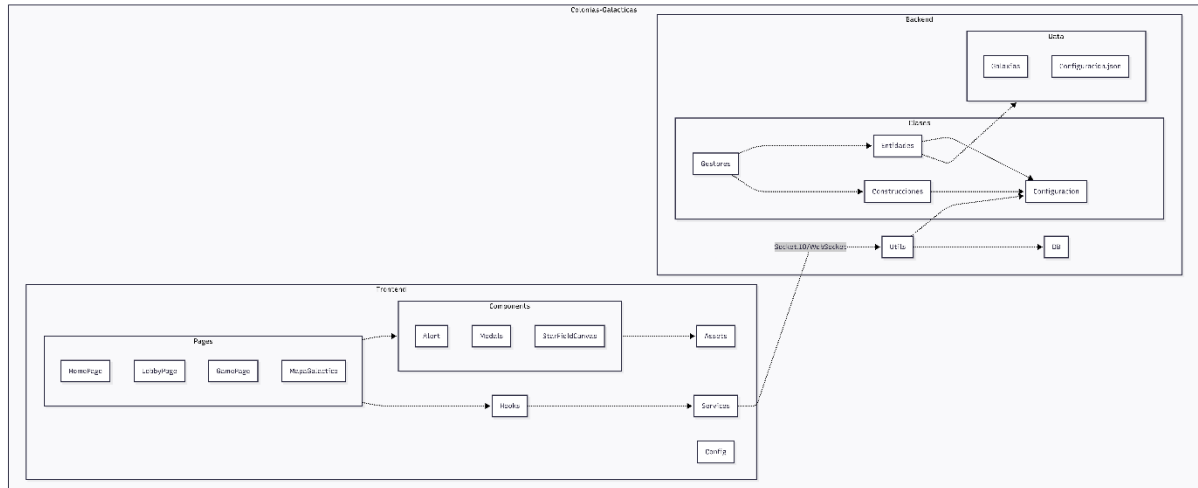
Descripción general del problema planteado por el proyecto.

Características principales:

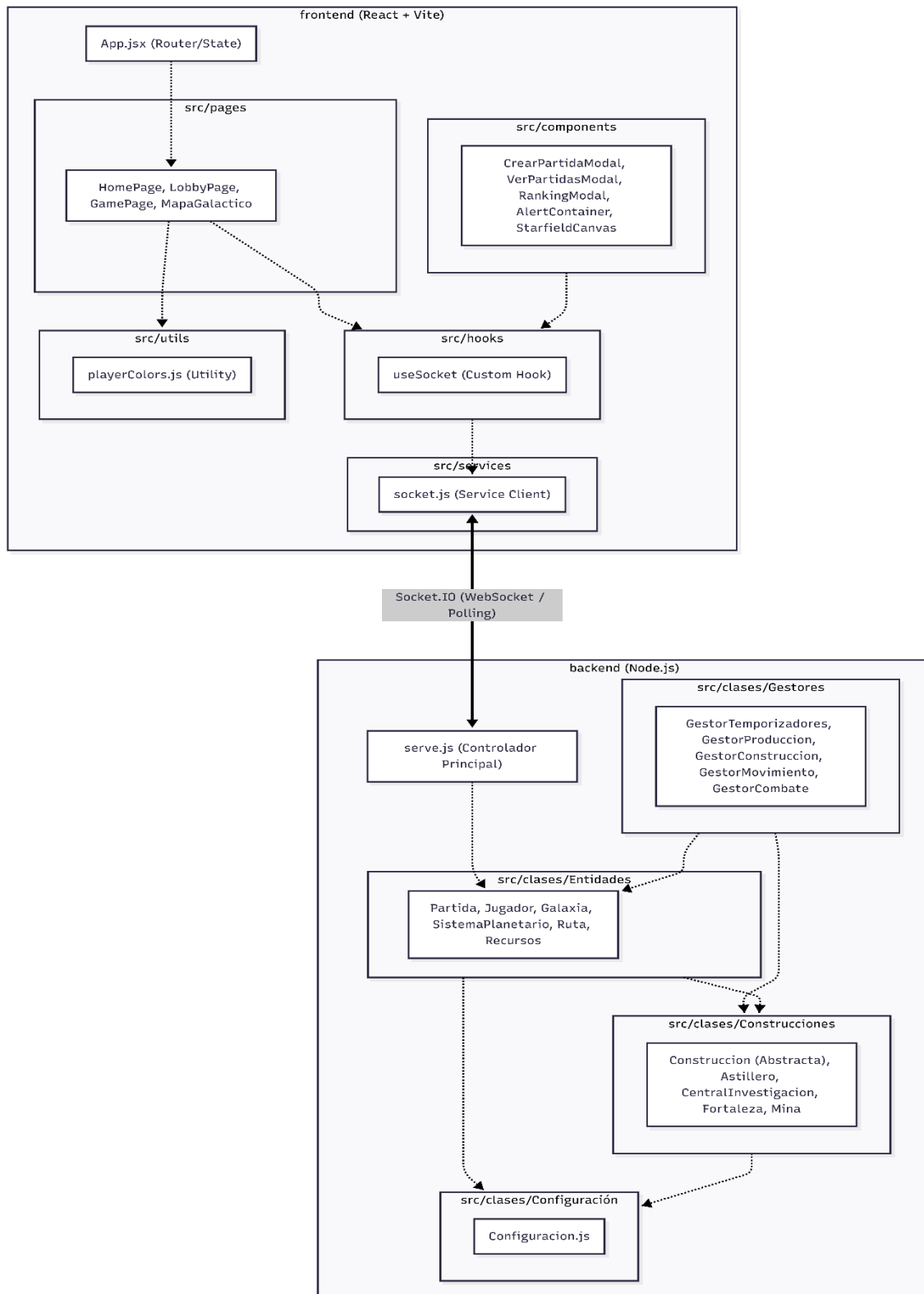
- Universo galáctico basado en grafos.
- Sistemas planetarios conectados mediante rutas espaciales.
- Creación y administración de partidas.
- Producción de recursos.
- Construcción de infraestructura.
- Movilización de flotas.
- Conquista de sistemas.
- Comunicación en tiempo real mediante WebSockets.
- Ranking histórico de partidas.

4. DISEÑO DEL PROGRAMA

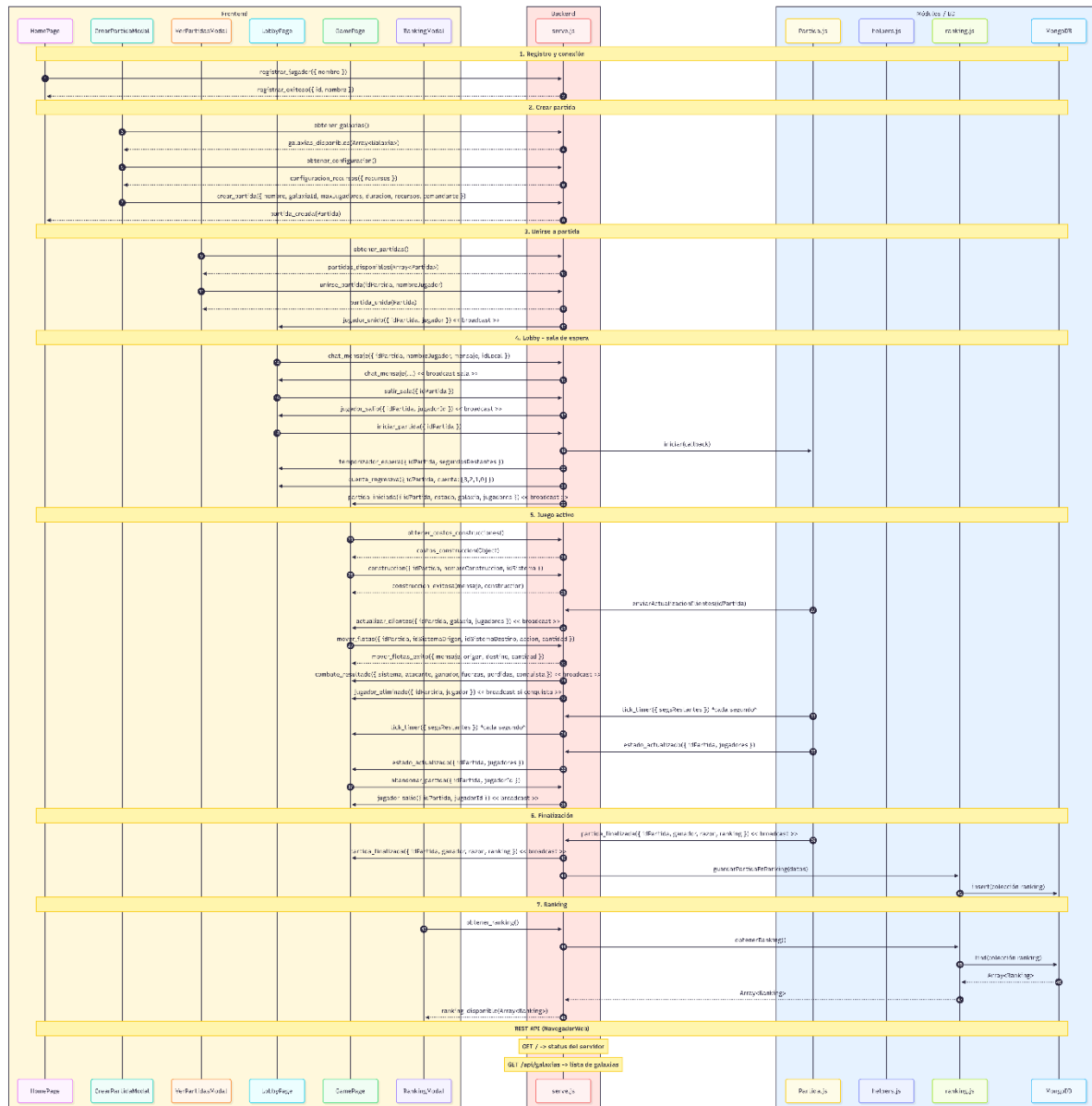
4.1 Diagrama de paquetes



4.2 Diagrama de distribución



4.3 Diagrama de comunicación



5. LIBRERÍAS USADAS

Backend

Librería	Función
express	Framework web para crear el servidor API REST
mongodb	Driver para conectar y operar la base de datos MongoDB
socket.io	Servidor WebSocket para comunicación en tiempo real
cors	Habilita peticiones entre dominios (frontend → backend)

dotenv	Carga variables de entorno desde el archivo .env
uuid	Genera IDs únicos para entidades

Frontend

Librería	Función
react	Framework de UI para construir la interfaz de usuario
react-dom	Renderiza componentes React en el DOM del navegador
react-router-dom	Enrutamiento y navegación en SPA
axios	Cliente HTTP para peticiones REST al backend
socket.io-client	Cliente Socket.IO para conexión en tiempo real con el servidor

Comunicación

Librería	Función
socket.io + socket.io-client	Comunicación bidireccional en tiempo real entre servidor y cliente
mongodb	Conexión y operaciones con la base de datos desde el backend
axios	Comunicación HTTP REST del frontend hacia el backend
cors	Permite la conexión segura entre dominios diferentes

6. ANÁLISIS DE RESULTADOS

6.1 Objetivos alcanzados

ID	Requerimiento	Implementado
RF-01	Autenticación por nombre	✓
RF-02	Crear partida	✓
RF-03	Recursos iniciales configurables	✓
RF-04	Cierre automático por inactividad	✓
RF-05	Listar partidas disponibles	✓

RF-06	Unirse a una partida	✓
RF-07	Sala de espera con cuenta regresiva de inicio	✓
RF-08	Carga de galaxia desde archivo JSON	✓
RF-09	Galaxia predefinida	✓
RF-10	Tipos de planeta y producción de recursos	✓
RF-11	Mapa galáctico interactivo	✓
RF-12	Actualización en tiempo real del mapa	✓
RF-13	Producción automática de recursos	✓
RF-14	Recolección de recursos por Centrales de investigación	✓
RF-15	Construcción de instalaciones	✓
RF-16	Construcción de flotas mediante Astilleros	✓
RF-17	Movilización de flotas	✓
RF-18	Combate automático	✓
RF-19	Conquista de sistemas libres	✓
RF-20	Condiciones de finalización de partida	✓
RF-21	Cálculo de puntuación final	✓
RF-22	Estadísticas de partida	✓
RF-23	Ranking histórico de partidas	✓
RF-24	Comunicación en tiempo real mediante WebSockets	✓
RF-25	Archivo de configuración del sistema	✓
RF-26	Asignación de planeta base inicial	✓

6.2 Objetivos no alcanzados

Todos los objetivos fueron implementados.

6.3 Razones por las cuales no se alcanzaron los objetivos

Todos los objetivos fueron implementados.

7. DESCRIPCIÓN DE LOS PRINCIPALES ALGORITMOS

7.1 Carga de galaxias

Objetivo: Cargar la configuración de una galaxia desde un objeto JSON, incluyendo sistemas planetarios y rutas entre ellos.

Entradas: Objeto JSON con la configuración de la galaxia (nombre, sistemas, rutas)

Salidas: Galaxia inicializada con sistemas y rutas cargados

Descripción: El método cargarDesdeJSON() en la clase Galaxia procesa un objeto JSON para inicializar la galaxia. Primero carga los sistemas planetarios mediante cargarSistemasDesdeJSON(), que crea instancias de SistemaPlanetario con sus IDs, nombres y tipos. Luego carga las rutas mediante cargarRutasDesdeJSON(), que crea instancias de Ruta conectando pares de sistemas por sus IDs.

7.2 Producción de recursos

Objetivo: Generar recursos periódicamente para todos los sistemas controlados por jugadores.

Entradas:

- Configuración del ciclo de producción (segundos)
- Estado de la partida (debe estar iniciada)
- Sistemas planetarios con propietario y estado 'controlado'
- Instalaciones en cada sistema

Salidas:

- Recursos agregados a los jugadores
- Total producido actualizado en cada sistema
- Evento de producción emitido vía callback

Descripción: El método producirRecursos() en GestorProduccion se ejecuta en intervalos regulares definidos por la configuración. Itera sobre todos los sistemas de la galaxia, verifica si tienen propietario y están en estado 'controlado', calcula la producción total del sistema mediante obtenerProduccionTotal(). Este método suma la producción base del sistema (según su tipo de planeta) más la producción adicional de cada Central de Investigación presente en

las instalaciones del sistema. Agrega los recursos al propietario del sistema y actualiza el contador de producción del sistema. Finalmente emite un evento de producción

7.3 Construcción de infraestructura

Objetivo: Validar y construir edificaciones o unidades en sistemas planetarios controlados por el jugador.

Entradas:

- `datosConstruccion`: Objeto con `nombreConstruccion` e `idSistema`
- `socketId`: ID del socket del jugador solicitante

Salidas: Objeto con resultado: `success` (boolean), `mensaje/error`, construcción creada, sistema, recursos restantes

Descripción: El método `construir()` en `GestorConstruccion` valida que el jugador exista, que el sistema sea válido y que sea propietario del mismo. Crea la construcción correspondiente (Mina, `CentralInvestigacion`, Fortaleza o Astillero) mediante `crearConstruccion()`. Verifica que el jugador tenga recursos suficientes mediante `verificarRecursos()`. Si todo es válido, resta los recursos al jugador y agrega la construcción al sistema (astilleros van a lista separada). Retorna el resultado con detalles de la operación.

7.4 Movilización de flotas

Objetivo: Mover astilleros entre sistemas planetarios, validando rutas y resolviendo combates si es necesario.

Entradas:

- `astilleros`: Array de astilleros a mover
- `sistemaDestino`: Sistema planetario de destino
- `callbackEvento`: Función para emitir eventos

Salidas: Objeto con `exitoso` (boolean), `errores` (array), resultado del combate si aplica

Descripción: El método `moverAstilleros()` en `GestorMovimiento` primero valida que exista una ruta válida entre origen y destino usando BFS (`encontrarCamino()`). Si el destino tiene propietario diferente al atacante, verifica si puede atacar y resuelve el combate mediante

GestorCombate.resolverCombate()). Si hay conquista, actualiza el propietario del sistema. Si no hay combate, simplemente mueve los astilleros al destino. Si el destino no tenía propietario, se asigna al jugador. Emite eventos de movimiento y conquista según corresponda.

7.5 Conquista de sistemas

Objetivo: Resolver combates entre atacantes y defensores, determinando conquistas y aplicando pérdidas.

Entradas:

- sistema: Sistema planetario atacado
- astillerosAtacantes: Array de astilleros atacantes
- jugadorAtacante: Jugador que ataca
- callbackEvento: Función para emitir eventos

Salidas: Objeto con: atacante, sistema, fuerzaAtacante, fuerzaDefensor, ganador, perdidasAtacante, perdidasDefensor, conquista (boolean)

Descripción: El método resolverCombate() en GestorCombate calcula fuerza de ataque (cantidad de astilleros) y fuerza de defensa (astilleros estacionados + minas/3 + fortalezas*2). Si el atacante gana, aplica conquista: cambia propietario, elimina instalaciones excepto Centros de Investigación, y estaciona astilleros restantes. Si el defensor gana, aplica pérdidas: primero astilleros, luego minas en grupos de 3, luego fortalezas. En empate, se destruyen todos los astilleros y se conservan solo Centros de Investigación. El desempate se define por minas residuales.

7.6 Cálculo de ranking

Objetivo: Calcular puntajes finales de jugadores basándose en sistemas controlados, recursos acumulados e infraestructura construida.

Entradas:

- Jugadores de la partida
- Sistemas de la galaxia
- Recursos de cada jugador
- Instalaciones de cada jugador

Salidas:

- Jugadores ordenados por puntaje descendente
- Ranking con posición, puntaje, nombre y estadísticas

Descripción: El método `calcularPuntajes()` en `Partida` calcula el puntaje de cada jugador mediante `calcularPuntajeJugador()`, que suma tres componentes: `calcularPuntajeSistemas()` (5000 puntos por sistema controlado), `calcularPuntajeRecursos()` ($\text{minerales} * 1 + \text{energía} * 2 + \text{cristales} * 3$), y `calcularPuntajeInfraestructura()` ($\text{Fortalezas} * 100 + \text{Centros} * 150$). Luego ordena los jugadores por puntaje descendente mediante `ordenarJugadoresPorPuntaje()`. El método `obtenerRanking()` retorna el ranking con posición, puntaje, nombre y estadísticas de cada jugador.

8. BITÁCORA

Registro de commits en el repo: <https://github.com/MaikelFn/Colonias-Galacticas>